AI Research Workspace —— 网页测试岗面试问答手册

> 面向网页/游戏测试岗的技术面试准备。所有答案都基于本仓库的真实实现， 不是通用模板。括号里的 `文件:行号` 是证据，被追问时可以直接翻代码。
>
> **项目一句话**：FastAPI + LangGraph 的研究工作台，前端 React 18 + Vite 5， 9 个页面、7 个工作流阶段、225 个后端测试、单端口托管部署。

目录

- 第 0 部分：开场自我陈述
- 第 1 部分：产品理解与测试设计
- 第 2 部分：功能测试深挖
- 第 3 部分：异常与边界场景
- 第 4 部分：兼容性、浏览器与多端
- 第 5 部分：性能、并发与资源
- 第 6 部分：网络与实时性（游戏测试核心）
- 第 7 部分：可用性、UI 与无障碍
- 第 8 部分：自动化与工程化测试
- 第 9 部分：线上问题排查与日志
- 第 10 部分：反问面试官
- 附：面试前速记卡

第 0 部分：开场自我陈述（30 秒版）

问：这个项目讲一下。

> 这是一个 AI 研究工作台。用户输入一个研究问题，后端用 LangGraph 编排一条 7 节点的工作流 ——拆解任务、定计划、检索证据、分析、核查，最后在写报告前**中断等人工确认**。 前端 React 单页应用，9 个页面，明暗双主题。
>
> 我在这个项目里主要做三件事：**接口层统一**（所有请求走一个 `api/client.ts`，统一信封解析、 超时、错误翻译）、**状态一致性**（长任务的进度、刷新恢复、重复提交拦截）、以及**视觉层可验证** ——把「好不好看」变成能用脚本断言的对比度数值。
>
> 我把它当成一个**有状态、长耗时、可中断、可恢复**的 Web 系统来测，这三点和游戏测试里 「客户端与服务端状态同步」的思路是同一套。

备注（为什么这样答）：

- 30 秒内讲完，不要展开技术细节，等对方追问。
- 「有状态、长耗时、可中断、可恢复」这五个词是钩子，主动抛出来引导对方往 **状态同步 / 断线重连 / 幂等** 这些方向问——这些都是你准备好的强项。
- 一定提「测试」而不是只提「开发」。面试官记不住技术栈，但会记住「这个人懂测试」。

第 1 部分：产品理解与测试设计

Q1.1 你会怎么给这个项目设计测试?

标准答法:

我会分五层设计，优先级从高到低:

层级	测什么	本项目的具体抓手
接口层	每个端点的正常流、参数边界、错误码	27 个接口，信封格式统一 `{success, data, error}`
状态机层	状态流转的合法与非法路径	工作流 7 状态：`running → awaiting_approval → completed/cancelled/failed`
端到端层	真实用户路径	提问题 → 等流水线 → 批准 → 拿到报告
前端交互层	加载态、空态、错误态、重复点击	4 类状态组件：`LoadingState/EmptyState/ErrorState/ErrorBoundary`
视觉与可访问层	对比度、响应式、键盘可达	主题色集中在 `tokens.css`，对比度有实测基线

深挖追问：「状态机层怎么测？」

> 状态机的测试重点是**非法转移**。合法路径好测，非法路径才会暴露竞态。 我在这个项目里写了 3 条专门的测试： ① `test_double_approval_is_rejected` —— 连点两次「批准」，第二次必须 409， 不能产生两份报告； ② `test_approval_persists_running_status_immediately` —— 批准后立刻查状态， 不能还停在 `awaiting_approval`； ③ `test_status_is_never_awaiting_approval_while_executing` —— **执行过程中** 查状态绝不能返回 `awaiting_approval`。
>
> 第 ③ 条是从线上 bug 反推出来的：原先用 `snapshot.next` 非空来判断「等待确认」， 但 LangGraph 执行中途 `next` 也是非空的，于是研究跑着跑着状态就漂移成 `awaiting_approval`。修法是改用显式业务标记 `approval_granted` 推导状态， 不依赖图内部位置。

加分点：主动说「我测的是非法转移，不是合法路径」——这句话能立刻区分出 「跑过用例」和「设计过用例」的人。

Q1.2 这个项目有哪些核心测试场景？你觉得最该测的是哪几个？

标准答法：

最该测的三个，我会排成这样：

① 中断-恢复一致性（最高优先级） 系统有个强制人工确认的中断点（`interrupt_before=["write"]`）。 这意味着存在「状态持久化 + 恢复」的一致性问题：

- 批准后刷新页面，确认框不能重新弹出来（否则用户会重复批准）
- 批准后进程重启，状态和报告必须还在（数据在 `agent_runs` 表落库）
- 终态（`completed` / `cancelled` / `failed`）再调 resume 必须 409，不能复活

② 证据可溯源性 这是产品的核心卖点。测试断言不只是「报告出来了」，而是：

- 报告里的每个结论都挂了 `[证据N]`
- 每个证据能追到来源（知识库某文档名 / 联网某网页标题）
- **一个证据编号都没标时，程序必须主动在局限说明里写「未经证据核对」**—— 这条是防「假装没问题」的，也是我重点测的负向用例

③ 检索质量的判据切换 知识库用的是本地哈希向量，没有语义能力。这带来一个反直觉的测试结论：

> 绝对相关度阈值在这里**切不开**。实测数据：

查询	最高分	实际相关性
向量数据库选型对比	0.587	相关
RAG 检索增强实践	0.441	相关
深度学习模型部署与量化	0.473	**完全无关**

无关内容的分数比相关内容还高，所以最终把判据从「分数够不够高」换成 「检索能力够不够格」：无语义能力时知识库直接不参与，退回联网检索， 并在时间线上写明原因。

备注：第 ③ 条是这个项目最有含量的测试设计，**一定要主动讲。** 它证明你不只是执行用例，而是能根据实测数据推翻一个看似合理的方案。 面试游戏测试岗时，这就是「设计测试用例」能力的最好证据。

第 2 部分：功能测试深挖

Q2.1 深度研究页面的完整测试点有哪些？

标准答法：

我会按「输入 → 执行 → 中断 → 恢复 → 产出」五段拆：

输入段

- 问题为空 / 纯空格 / 只有标点 → 应有前端校验，不能发出空请求

- 超长问题（如 5000 字）→ 截断策略是什么？会不会撑爆输入框
- 特殊字符：`<script>alert(1)</script>`、SQL 片段、emoji
- 重复提交：连点「开始研究」按钮 → 幂等还是起两个 run？（**关键点**）

执行段

- 流水线 7 个阶段依次点亮，步骤条不跳步、不倒退 （图里实际有 8 个 `add_node`：7 个业务阶段 + 1 个 `fail` 失败终态节点）
- 轮次显示 `Research ×N`，N 不超过 `max_iterations`
- token 消耗递增且与实际调用量吻合
- 运行中切页再切回，进度不丢

中断段

- 状态停在 `awaiting_approval`，确认框可见
- **此时刷新页面，确认框不该重新弹出**（除非本来就没批准过）
- 点「拒绝并终止」→ 状态 `cancelled`，不生成报告
- 中断期间后端进程重启 → 恢复后状态仍是 `awaiting_approval`，不是 `failed`

恢复段

- 批准后立刻查状态 → 已是 `running` 不是 `awaiting_approval`
- 批准后再次批准 → 409
- 反馈意见填了要生效（写进报告的局限说明）
- 拒绝时也允许填意见

产出段

- 报告含 `sections` 和 `limitations` 两个部分
- 结论挂了证据编号，且编号在证据面板里能点进去
- 报告能导出/下载
- 报告在「研究报告」历史列表里能查到，能点回看

深挖追问：「你怎么测幂等？」

> 这个项目没有靠前端按钮 disable 来防重复提交，而是靠**后端状态机**： 同一个 thread 的终态再调 resume 返 409。所以我的测试方式是 **并发发起两次 resume，断言只有一次成功、另一次 409，且 `agent_runs` 表里只有一条记录**（不是只断言 HTTP 状态码，还要断言数据没有重复—— 状态码对了但数据写了两份，是很常见的漏测点）。

备注：括号里的表名（`agent_runs`）和状态码（409）能说出来，可信度立刻不同。 如果被问到具体怎么发并发请求：`curl` 两个后台进程，或 Python 里 `asyncio.gather` 两个 `requests.post`。

Q2.2 智能体工作台和深度研究有什么区别？测试上要注意什么？

标准答法：

智能体是**自主工具调用循环**——模型自己决定调哪些工具（联网搜索 / 抓网页 / 计算 / 查知识库），没有固定的 7 节点路径。测试重点因此完全不同：

维度	深度研究	智能体
路径	固定 7 节点	模型自主决定，不确定
断言	状态机 + 报告结构	只能断言「有答案 + 轨迹合法」
终止条件	图跑完	达到 `max_steps` 或模型主动收尾
主要风险	状态漂移	死循环 / 工具调用不收敛

具体测试点：

- 工具调用轨迹里，每个 step 的工具名、参数、返回都要能被看到和回放
- 达到最大步数要**优雅终止**，不是卡死
- 工具全部失败时要给出明确失败原因，不能是空白页
- **历史记录要持久化**：跑完不能丢，能点回看当时的答案与轨迹
- 轨迹的每条记录都要有来源标注（哪个网页、哪个文档）

深挖追问：「自主性强的系统怎么测？」

> 核心思路是：**把不可测的输出换成可测的中间状态。** 模型最终答案没法用固定断言测，但「工具调用了几次、每次调了什么、 参数是否合法、返回是否被正确引用」是可以断言的。
>
> 所以我会断言**轨迹的结构合法性**而不是**答案的内容正确性**—— 这是我做这类系统测试时总结出来的通用方法。

第 3 部分：异常与边界场景

Q3.1 接口出错时，前端怎么表现？测试过吗？

标准答法：

这个项目的前端错误处理是**分层**的，我按层测：

第 1 层：网络层失败（后端没起 / 断网） → 统一抛 `ApiClientError`， `code = network_error`， 文案是「无法连接后端（实际地址），请确认服务已启动」。 **测试点**：文案里显示的地址不能是空的——同源部署时 `API_BASE_URL` 是空字符串， 早期版本拼出来是「无法连接后端（），…」这种空括号，直接被测出来。

第 2 层：响应不是 JSON（网关返回 HTML 错误页） → 这是最容易被漏测的一类。真实场景是反向代理超时后返回 502/504 的 HTML 页面。 早期实现直接 `JSON.parse` 抛裸 `SyntaxError`，用户看到的是 「Unexpected token '<'」这种没法理解的报错。 → 现在先把 body 读成文本再解析，识别出是不是 HTML，给出可诊断的文案， 并带上响应片段（120 字符）。

> 这一点对游戏测试岗同样关键：**服务器返回 502 的 HTML 错误页**，如果客户端只弹「Unexpected token '<'」，玩家根本不知道是网络问题还是服务器炸了。

第 3 层：业务错误（HTTP 200 但 `success=false`）→ 后端保证 `success=false` 时一定带 `error` 字段，取 `error.code` 做分支，`error.message` 展示。

第 4 层：特殊业务码 → `daily_budget_exceeded`（每日运行额度用尽）时，后端只回 `reset_at_ms` 时间戳（它不知道也不该猜客户端时区），前端负责翻译成「今天 23:59 恢复」这种人话。

深挖追问：「为什么不让后端直接返回人话？」

> 后端返回结构化数据、前端做本地化，是有意为之：后端可能跑在 UTC 时区的服务器上，它不知道用户在哪，猜一个时区就可能给出错误的时间。前端拿 epoch 毫秒用本地时区渲染，结果一定是对的。这个设计在游戏客户端里也是一样的—— **服务器发时间戳，客户端按本地时区显示。**

Q3.2 限流和配额这块测了吗?

标准答法:

测了，后端有**两个独立的限流**:

① 请求级限流（`RateLimitMiddleware`）

- 纯 ASGI 实现，命中时**直接返回 429，根本不进入路由**
- 响应带 `Retry-After` 头
- 有测试专门验证「不匹配的路径不受限流影响」——防止限流规则误伤只读接口

② 每日运行额度（`DailyRunBudgetMiddleware`）

- 按**客户端 IP** 统计「**新建运行**」的次数，不是所有请求
- 超限返回 429，`error.code = daily_budget_exceeded`
- 计数器**只统计成功的运行**——失败的不该占额度
- **跨天自动重置**（有专门测试用 monkeypatch 把日期推过去验证）
- 过期记录要清理（`_prune`），否则内存无限增长

深挖追问：「限流返回 429 时前端怎么处理？」

> 前端有专门的分支：`ApiClientError` 带 `code` 和 `status`，UI 读到 `daily_budget_exceeded` 就渲染成「额度已用尽」的特殊提示，而不是通用的「请求失败」。同时 `Retry-After` 头会带重试等待秒数。
>
> 另外有个细节：限流中间件是**纯 ASGI** 而不是 `BaseHTTPMiddleware` —— 后者会在限流命中时也先把请求体读完，白白消耗连接。限流要尽快拒绝，就不该经过完整的请求解析。

备注：这一问在游戏测试岗很常见（「服务器限流了客户端怎么表现」）。能答出「纯 ASGI vs BaseHTTPMiddleware」的差别，说明你真的看过代码，而不是背过「429 就提示重试」。

第 4 部分：兼容性、浏览器与多端

Q4.1 前端做了哪些浏览器适配？测试矩阵怎么设计？

标准答法：

先说实现，再给矩阵。

实现上的关键点：

- **响应式断点只有 4 个**（`frontend/src/styles/responsive.css`）： - `≥1200px`：顶栏 9 个导航平铺在 3 组内，组标签可见 - `1024–1199px`：隐藏组标签，只留组间竖线 - `768–1023px`：顶栏只保留核心 3 项（`dashboard` / `workflow` / `reports`）+「更多」下拉 - `<768px`：汉堡按钮 + 移动抽屉
- **导航是三个容器共用同一份渲染代码**（顶栏 / 下拉 / 抽屉）， 零分支——所以不会出现「下拉里能点、抽屉里点不了」这种不一致
- **不引入路由库**，用 URL hash 自己解析；页面级 lazy loading， 首屏只下载当前页代码
- 走 `hash` 路由还有个副作用好处：**兼容后端 SPA fallback**（任何路径都回落 `index.html`），且天然支持刷新、分享、后退

测试矩阵我会这样设计：

维度	取值
浏览器	Chrome / Edge / Firefox / Safari（**必测 Safari**，Mac 用户占比高）
视口	1920 / 1440 / 1280 / 1024 / 768 / 375
主题	浅色 / 深色
缩放	100% / 200%（浏览器缩放是最容易漏的）

深挖追问：「为什么缩放要单独测？」

> 因为 200% 缩放等价于视口宽度减半，会触发断点切换，很多布局问题只在缩放下暴露： 顶栏可能从「9 项平铺」跳到「3 项 + 下拉」，如果下拉里的项没有全部渲染出来， 某些页面就变成**不可达**了。测这个要配合「能否用键盘 Tab 到每一项」一起看。

Q4.2 深色主题怎么测？会不会有文字看不清？

标准答法：

深色主题**不是把颜色反过来**，是独立的一套令牌。所以我的测试方式是： 先验证令牌本身，再验证渲染结果。

第一层：令牌断言 `frontend/scripts/probe-visual-theme.cjs` 用 CDP 进真实浏览器读 `getComputedStyle`，断言每个前景/背景组合的对比度。

第二层：对比度基线 配色不靠肉眼定，全部脚本实测：

- 浅色底正文 ≥ **5.0:1**
- 深色底正文 ≥ **6.5:1**
- 主按钮 5.17:1

这里有个真实的坑：深色主题下的渐变按钮，最初配的是白字， 但**渐变的末端恰好是文字所在那段**，最差只有 1.81:1 —— 按钮几乎看不见。 改法是反过来：亮渐变 + 近黑字（`#5b93ff → #5093f8 → #22d3ee`）， 三档实测最差 6.44:1。

> **测试心得**：渐变背景的对比度不能只看一个值， 要按渐变的**起止两端分别算**，文字压在哪一段就用哪一段的色值。

第三层：像素级验证 `probe-visual-zoom.cjs` 用 2x 像素密度截图，然后**手写 PNG 解码** （zlib inflate + 5 种 filter 反算）做像素采样，验证氛围网格真的落到像素上 （实测间距 62–64 设备像素，与 32px 栅格的设计值吻合）。 这一步是为了防止「样式写了但被别的规则盖掉」。

第四层：hover 态 断言 hover 态的「文字 vs 自身背景」≥4.5:1——因为 hover 态是独立选择器， 很容易在改配色时漏改。

深挖追问：「探针脚本和普通的单元测试有什么区别？」

> 单元测试跑在 JSDOM 里，没有真实布局引擎，`getComputedStyle` 返回的是默认值， 渐变、伪元素、`backdrop-filter` 全都测不出来。探针是真浏览器 + CDP， 能量真实几何、读真实计算样式、截真实像素。两者是互补的：单元测业务逻辑，探针测「样式有没有真生效」。

第 5 部分：性能、并发与资源

Q5.1 这个系统的性能瓶颈在哪？你测过什么？

标准答法：

先说结论：**瓶颈不在前端渲染，在模型调用和网络等待。**

核心事实：一次深度研究要跑 2–5 分钟，绝大部分时间是等 LLM 和搜索接口。 所以前端的性能目标不是「让操作变快」，而是**让等待变得可感知。**

我做过的性能相关设计：

① **接口立即返回，不阻塞** 工作流跑在后台任务里，`POST /api/graph/research` 立刻返回 `running`。 如果同步等整条流水线，必然撞上网关约 60 秒的超时，用户只看到一个 504。

② **前端请求超时 300 秒** 注释里写明：启动一次研究要 1–2 分钟，给足余量； 但也不能无限等，否则网络断开时 UI 会永远停在「执行中」。

③ **页面级 lazy loading** 9 个页面用 `React.lazy` 按需加载，首屏只下载当前页的代码； 另外在 `requestIdleCallback` 里预热「深度研究」这个推荐入口，消掉首次点击的加载感。

④ **事件增量拉取** 轮询通道是 `?after=<lastId>` 增量拉，不是全量重传。

测试手段：

- 测接口响应时间时区分 **TTFB** 和 **总时长**
- 长任务不阻塞 UI：运行期间应该能切页、能看历史，不冻结
- 内存/句柄泄漏：反复进出页面 20 次，看有没有残留的 `EventSource` 连接、 定时器没清。这是前端测试的经典检查项

深挖追问：「怎么测有没有连接泄漏？」

> 这个项目有个具体问题：SSE 用 `EventSource`，切换页面时不关闭会一直挂着。 所以我会打开 Chrome DevTools 的 Network 面板，看切页后 `text/event-stream` 的请求是否还在 pending； 或者在代码层面查 `useEffect` 的 cleanup 函数是否调用了 `source.close()`。
>
> 另外 `api/client.ts` 里的 `withTimeout` 同时监听**外部 signal** 和**超时定时器**， 返回 `cleanup` 去做 `clearTimeout` + 移除监听——不清理的话， 每次请求都会留一个定时器，长时间使用后定时器队列会越积越多。

Q5.2 并发场景测过吗？同时开两个研究会怎样？

标准答法：

会怎样取决于**数据库层和进程模型**：

数据库层：SQLite 在这个项目里承担了两类东西—— 运行记录（`agent_runs`、`events`）和知识库（`documents`、`document_chunks`）。 SQLite 是单写多读，并发写会锁库。

进程模型：部署是**单进程**的，因为**配额记账依赖单进程内存状态** （按客户端 IP 统计当日新建运行数，存在内存字典里）。 如果加 `--workers` 起多进程，每个进程有自己的计数器， **限流就会被绕过 N 倍。**

> 这是一个典型的「性能与正确性二选一」的场景： 多 worker 能提升吞吐，但会让限流失效。 所以选择单进程 + 靠事件循环处理并发。

测试点：

- 同时发 3 个 `POST /api/graph/research`，断言返回 3 个**不同的** `thread_id`
- 三个 run 的事件流**不能串**—— `subscribe(thread_id)` 必须只收到自己那条
- 事件表的 `list_since(thread_id, after_id)` 要按 thread 隔离
- 并发 resume 同一个 thread → 只能一个成功

深挖追问：「事件流怎么保证不串？」

> 事件总线是**按 thread_id 分频道**的： `EventBus.subscribe(thread_id)` 返回一个 专属的 `asyncio.Queue`， `emit(thread_id, ...)` 只往对应队列投递。 另外事件是**先落库再广播**的（`events` 表），所以就算订阅者晚连上， 也能用 `last_event_id` 从历史里补齐——这解决了「刷新页面就丢事件」的问题。

备注：这一问很能区分候选人的深度。绝大多数人只会答「测并发没问题」， 能讲清楚「限流计数器在内存里所以不能多进程」才是真的懂。

第 6 部分：网络与实时性（游戏测试核心）

> 这一节是**游戏测试岗最看重**的部分，务必准备透。

Q6.1 实时进度是怎么实现的？网络不好会怎样？

标准答法：

理想方案是 SSE（Server-Sent Events），单向推送，正好匹配「后端推进度给前端」。

但这个项目遇到一个很典型的问题：

> 部署环境的**反向代理会缓冲** `text/event-stream`。 表现是：连接能建立，但**连响应头都不下发**， 浏览器的事件连接永远停在 `CONNECTING`， `onopen` 和 `onerror` **都不触发。**
>
> 所以进度条一开始不自己走，得手动点刷新。

解决思路是「双通道 + 自动降级」：

1. 后端补一个普通 JSON 的事件拉取接口： `GET /api/graph/runs/{id}/events?after=N` —— 普通 JSON 不会被任何网关缓冲
2. 前端加**看门狗**： `SSE_OPEN_TIMEOUT_MS = 6000`， 6 秒内 SSE 没 `open`，就关掉那条连接改用轮询（ `POLL_INTERVAL_MS = 2500`）
3. 轮询期间每 3 秒（ `SNAPSHOT_POLL_INTERVAL_MS = 3000`）回拉一次运行快照， 让步骤条、轮次、token 自己往前走
4. **本地直连时仍然走 SSE**（本地没有代理缓冲）

> 这个设计和游戏客户端的**「长连接 + 心跳 + 降级轮询」**是同一套思路： 不能假设网络永远通畅，主通道不通要有备用通道， 而且降级要**自动**、对用户无感。

深挖追问 1：「怎么测出是代理缓冲，而不是后端没实现？」

> 分离测试，两步： ① **本地直连**：起后端， `curl -N` 直连 SSE 接口，能看到数据流在刷 → 后端没问题 ② **走代理**：从线上域名请求，连接建立但没有响应头 → 问题在代理层
>
> 我还会看一个细节：**响应头都不下发**是缓冲的典型特征。 如果是后端逻辑问题，通常至少能收到 200 和头，只是没有 body。 这个区分能省掉很多无效的后端排查。

深挖追问 2：「降级的时候怎么不把连接数撑爆？」

> 降级时会**先 `close()` 掉那条永远连不上的 SSE**。 原因是 HTTP/1.1 下每个域名只有 **6 个并发连接**， 让一条卡死的 SSE 挂着会挤掉后续的轮询请求。
>
> 同样的道理， `switchStream()` 在切换到新 thread 时会关掉其它所有连接， 避免连接数无限增长——**连接数是有限资源，测试时要专门盯**。

深挖追问 3：「切到别的页面，进度会不会断？」

> 不会，而且这是专门改过的。
>
> 最初连接由 `useRunEvents` 这个 hook 独占，组件卸载（切导航）就 `close()`， 于是页面文案写的「你可切到其他模块同步查看，进度不会中断」**根本不成立**—— 一切走就断流，回来只剩手动刷新。
>
> 现在改成**模块级连接管理器**：
>
> - 连接生命周期与组件挂载周期**解耦**
> - 组件只注册/注销「监听器」，**不拥有连接**
> - 新加入的监听器会先拿到已缓存的事件（ `eventCache` ）， 所以切回来时进度是完整的
> - 连接只在**终态事件**、显式 `closeStream()` 或切到新 thread 时关闭
>
> 顺带一个细节：取消订阅时**刻意不**关闭连接—— 组件卸载不应该中断后台运行。

深挖追问 4：「重复事件怎么办？」

> 事件可能因重连或补发而重复，所以有两层去重： ① 底层 `ingest()` 里按事件 id 查缓存，已存在的直接 `continue` ② React 侧 `setEvents` 里再判一次 `prev.some(e => e.id === event.id)`
>
> 这对游戏测试来说是熟悉的场景：**网络重传导致的重复消息**， 客户端必须幂等，否则界面上会出现两条一样的记录。

Q6.2 断网重连后，进度会不会丢?

标准答法：

不会丢，靠两个机制：

① **事件先落库再广播** `events` 表存了每个事件，带自增 id 和 `thread_id`。 订阅时传 `last_event_id`，服务端用 `list_since(thread_id, after_id)` 补发历史事件。

② **前端记录 lastId** 轮询接口返回 `{events, last_id}`，前端拿 `last_id` 作为下次的 `after` 参数。 所以即使断网期间漏掉了几条事件，重连后也能补齐——**不会丢，也不会重复。**

测试方法（这是关键，面试官会看你是不是真测过）：

步骤	操作	预期
1	正常启动研究，等到 `research ×2`	进度条走到第 2 轮
2	**DevTools → Network → Offline**	前端应显示断连，不崩、不清空已有进度
3	保持离线 10 秒	此时服务端仍在跑，事件继续产生
4	**恢复 Online**	进度**自动接上**，且不重复、不跳号
5	切到别的页再切回	进度仍在，不回到初始态

深挖追问：「断网期间用户点了批准会怎样？」

> 这是个真实的边界情况。断网时请求发不出去， 前端的 `fetch` 会走网络层失败分支，抛 `network_error`， UI 显示「无法连接后端」。**用户的输入不会丢**—— 因为表单状态在 React 里，没有提交成功就一直保留着，界面不会清空。 恢复网络后重新点提交即可。

深挖追问 2：「如果断网时后端已经完成了，用户回来看到什么？」

> 因为状态是**落库**的（`agent_runs` 表），回来后重新拉状态接口， 拿到的是真实终态 `completed` + 报告，而不是「还在运行」。 这里的关键是**前端不缓存状态真相**，每次都以服务端为准。

备注：这三条追问是本手册的核心。游戏测试岗几乎一定会问 「断线重连后状态对不对」「重复提交怎么处理」「弱网下玩家看到什么」， 答好这三条基本就过了。

Q6.3 重复提交 / 快速连点怎么处理?

标准答法:

分两层防:

前端层：按钮在请求进行中置为 disabled，防止连点。

后端层（真正兜底的）：**幂等靠状态机，不靠前端按钮。**

- 同一个 thread 的终态再调 resume → **409**
- 双击批准 → 第二次被 `test_double_approval_is_rejected` 拦住
- 每日额度只统计**成功的**新建运行

> **测试上的原则**：前端防重复只是体验优化，后端幂等才是正确性保证。 因为前端可能被绕过（直接调接口、多个标签页、网络重发）。 我测幂等一定**从接口层并发发起**，不靠点按钮。

深挖追问：「多标签页打开同一个研究，怎么处理？」

> 这是一个很实际的场景。两个标签页都打开了同一个 thread， 一个批准了，另一个还显示「等待确认」。刷新第二个标签页， 它拉到的状态已经是 `completed`，界面应该跟着更新成已完成， 而不是还弹着确认框等一个永远等不到结果的批准。

第 7 部分：可用性、UI 与无障碍

Q7.1 一个页面要展示「加载中 / 空数据 / 出错 / 有数据」四种状态，你怎么设计?

标准答法:

这个项目有 4 个对应的组件，我按优先级说:

状态	组件	关键点
加载中	`LoadingState`	支持 `list` / `card` / `table` 三种骨架形态，**骨架屏要跟最终内容形状一致**，否则数据出来时布局会跳
空数据	`EmptyState`	不能是空白，要给出「为什么空 + 下一步做什么」
出错	`ErrorState`	要区分「可重试」和「不可重试」
兜底	`ErrorBoundary`	组件渲染崩溃时兜住整页，不白屏

深挖追问:「空状态文案你会怎么写?」

> 知识库页空的时候，写的是「上传 PDF / Markdown / TXT，系统自动解析、切块、向量化， 把研究智能体在检索时引用」——**说清楚能做什么、下一步做什么**， 而不是「暂无数据」。
>
> 因为「暂无数据」对用户没有信息量，用户会以为自己操作错了。

深挖追问 2:「加载骨架屏怎么测?」

> 骨架屏的测试点不在「有没有转圈」，而在**布局稳定性**: 加载中列表是 3 行占位、数据回来是 5 行，页面高度会跳。 我会对比加载中和加载完成的截图，检查主体内容位置是否稳定。
>
> 另外有个实操坑：这个项目的截图脚本最初就是只等「根节点有子节点」， 结果**页面级 lazy 加载期间骨架屏已经挂上根节点了**，条件提前通过， 截出来是一张灰条。修法是显式等骨架屏消失（`.skeleton`）。 这个坑对做 UI 自动化的人很典型——**等待条件要等「内容就绪」，不是等「DOM 就绪」**。

Q7.2 无障碍这块做了多少?

标准答法:

不敢说「完整」，但**关键的部分做了**：

- **焦点可见**：`base.css` 里有统一的 focus ring 样式
- **语义标注**：19 处 `aria-label`、11 处 `aria-hidden`（装饰性图标要藏掉，否则读屏会念出一堆「图标」）、`aria-current="page"` 标记当前页、`aria-expanded` 标记折叠态、`aria-modal` 标记弹窗
- **动态内容播报**：2 处 `aria-live`，Toast 和加载状态能被读屏播报
- **动效可关**：`@media (prefers-reduced-motion: reduce)` —— 尊重系统「减弱动态效果」设置
- **键盘可达**：导航是原生 `button`，天然可 Tab；抽屉用 Esc 关闭；折叠卡片有 `aria-expanded` + `aria-controls`

深挖追问：「`aria-hidden` 用错会怎样？」

> 用在**可交互元素**上是错的。比如一个带图标的按钮，如果给按钮加了 `aria-hidden="true"`，读屏用户就**完全感知不到这个按钮存在**。正确做法是给**装饰性图标**加 `aria-hidden`，让按钮自己带 `aria-label`。这类问题肉眼看不出来，只能靠读屏器实测。

加分点（如果被问到提升建议）：

> 还缺的部分我知道：**完整的键盘导航顺序管理**（抽屉打开时焦点应该锁在抽屉内，关闭后焦点回到触发按钮，这是 focus trap + focus restore），以及**读屏器实测**（目前是用 `aria` 属性静态检查，没有真正用 NVDA/VoiceOver 跑过）。我会优先补 focus trap，因为它直接影响键盘用户的可用性。

备注：敢说「还缺什么」比只说「做得很好」可信度高得多。游戏测试岗有时也要做无障碍检查（部分出海产品有要求）。

第 7.5 部分：几个具体的小 Bug（考观察力）

Q7.3 弹窗为什么曾经不居中？

标准答法：

这是我觉得这个项目里最能说明「测试要往深了做」的一个例子。

现象：报告弹窗没居中，顶部被导航栏盖住。但代码上完全看不出问题——弹窗是 `position: fixed; inset: 0` 加 `place-items: center`。

排查方式：用 DevTools 协议在真实页面里**逐层扫祖先的计算样式**，量出来是这样：

```
TEXT
main.main → transform=matrix(1,0,0,1,0,0)   animation=page-in fill=both
```

根因：页面转场动画写了 `animation-fill-mode: both`。`page-in` 只有起始关键帧，但 `both` 让动画结束后**继续保留动画算出来的 transform**—— 值是恒等矩阵，**不是** `none`。

而只要一个元素的 transform 不是 `none`，它就成了所有 `position: fixed` 后代的**包含块**。 于是弹窗相对 `<main>` 定位（从导航栏下沿开始），而不是相对视口。

修复（两条都做）：

1. 填充模式改成 `backwards` —— 视觉完全没变，但动画结束后不再残留 transform
2. 弹窗同时用 **portal 挂到** `body` 上 —— 从结构上免疫这类祖先影响

改完的实测：四个视口下遮罩都是 `0,0,视口宽,视口高`，居中偏差 ≤1px。

测试心得（这句是加分点）：

> 「`position: fixed` 不等于「相对视口定位」」—— 只要祖先有 transform / filter / perspective / will-change， fixed 就会相对那个祖先。这个坑在游戏 UI 里也常见 （弹窗挂在带缩放动画的容器里）。

Q7.4 你还发现过什么隐蔽的问题?

标准答法：

举一个**前端换字体**的例子，因为它特别能体现「测到位了」：

换视觉方案时，我把令牌里的字体从衬线换成了无衬线，**但忘了改** `index.html` **里的字体外链**。 结果是：

- Inter **从未被下载**
- 全站静默回退系统字体
- 计算样式照样显示 `Inter`（声明还在），**肉眼看代码发现不了**
- 而且白白拉了两个用不到的字体，外链 CSS 阻塞首屏

怎么发现的：写探针脚本时加了断言—— `document.fonts.check('500 16px "Inter"')`，运行时发现返回 false。

这里还有个细节：一开始用 `document.fonts.check('16px "Inter"')` 断言，一直返回 false， 但 `check('600 16px "Inter"')` 返回 true。 原因是 `check()` 不带字重时**默认按 400 查**， 而外链只请求了 500/600/700 三个字重。

> **测试心得**：断言要按**真实使用条件**写，不能为了通过而放宽。 如果当时直接把断言改成「只要有一个字重加载就算过」， 这个缺陷就会被断言自己放过去。

顺带两个测试环境相关的坑（如果面试官问「你怎么搭测试环境」）：

- headless Chrome 缺 `--no-sandbox` 或 `--user-data-dir` 时，进程起来了但**端口不监听**， 表现为探针无输出、退出码还是 0 —— 静默失败最容易骗人
- Git Bash 会把 `/` 开头的参数当 MSYS 路径转换， `node probe.cjs url "/"` 会变成 `C:/Users/.../PortableGit/.../`， 探针访问到无效路由读回来的是浏览器错误页

第 8 部分：自动化与工程化测试

Q8.1 你们的自动化测试怎么做的?

标准答法:

分三层，各管各的:

层	工具	覆盖什么
单元/集成	pytest + FastAPI TestClient（225 个测试函数，237 个用例通过）	业务逻辑、接口契约、状态机、中间件
契约与守卫	pytest 里专门的「生产加固」测试文件	安全头、限流、每日额度、跨天重置、JSON 兜底
端到端/视觉	Node + CDP 直连 Chrome DevTools Protocol	真实浏览器的几何、计算样式、像素采样、跳转落地

为什么用 CDP 而不是 Playwright/Selenium:

这个项目**前端依赖只有 5 个**（react / react-dom / motion / react-markdown / remark-gfm）， 刻意不引测试框架。CDP 是 Chrome 调试协议本身的能力， 用 Node 22 内置的 WebSocket 直接连，**零新增依赖**， 而且要什么断言就自己量——读取计算样式、截指定区域的图、采样像素值， 比通用框架的 API 更自由。

具体有 4 个探针脚本（`frontend/scripts/`）：

脚本	测什么
`probe-visual-theme.cjs`	双主题令牌、渐变、玻璃层、字体**是否真被加载**、横向溢出
`probe-visual-zoom.cjs`	2x 像素密度 + 手写 PNG 解码做像素采样，网格落到像素、hover 对比度
`probe-modal-layout.cjs`	弹窗几何：遮罩是否铺满视口、居中偏差 ≤1px
`probe-evaluation-jump.cjs`	效果评估页跳转是否落到正确页面
`docs-screenshots.cjs`	文档用首屏截图（顺带也是一次渲染验证）

深挖追问：「手写 PNG 解码是不是过度设计了？」

> 目的是**验证氛围网格真的落到像素上。** 用 `getComputedStyle` 读 `background-size` 只能证明「样式声明了 32px」， 但证明不了「渲染出来就是 32px」——中间可能被 `background-size` 的其他规则、 或者父元素的 `transform: scale` 改掉。解 PNG 量像素是唯一能确认的办法。
>
> 顺带一个发现：`document.fonts.check()` 不带字重时按 400 查， 断言必须按真实字重写。

Q8.2 测试环境数据会互相干扰吗?

标准答法:

会，而且这个项目踩过很严重的坑。

历史记录里有 942 条运行记录，用户反馈「不就点了一次吗」。 查下来是：**测试只 mock 了模型和搜索，从来没隔离 SQLite 路径**—— 每跑一次测试就往真实数据库写几十条运行记录， 942 条里 923 条是测试写的。而且部署工具会把 `backend/` 整个目录打包上传， 于是本地测试垃圾原样上了线。

修法:

- 测试会话的**四个库路径全部指向一次性临时目录**
- 加一条专门的测试守着这件事，防止以后改回去

测试隔离的通用要点（可以展开讲）：

1. **数据隔离**：每个测试用独立临时目录 / 独立 DB，测完清理
2. **外部依赖隔离**：mock 模型和搜索接口（这是这个项目做的）
3. **时间隔离**：涉及「每日额度」「跨天重置」的测试要能控制时间， 用 `monkeypatch` 把日期推过去，而不是真的等一天
4. **状态隔离**：共享的单例（比如内存里的限流计数器、 `EventBus`）需要在测试间重置，否则测试之间有顺序依赖

加分点:

> 我还发现过**测试顺序依赖**：某些测试因为跑在前面把内存状态改坏了， 导致后面的测试失败，但单独跑又是过的。 这类问题在 CI 上特别烦，因为 CI 是全量顺序跑。 排查手段是 `pytest -p no:randomly` 固定顺序 + 二分定位。

第 9 部分：线上问题排查与日志

Q9.1 线上出问题了，你怎么定位?

标准答法:

第一步：确认问题范围

- 是所有用户还是个别? —— 看 `X-Request-ID`
- 是前端还是后端? —— 直接 curl 接口

第二步：拿到请求标识 这个项目每个请求都会带 `X-Request-ID`： 中间件会取请求头里的，没有就生成一个 12 位 uuid， **同时写进响应头**。所以用户报障时给一个 ID 就能串起整条链路。

第三步：分类错误 前端把所有失败统一包成 `ApiClientError`，带三个字段：

- `code`：机器可读的分类（`network_error` / `invalid_response` / `aborted` / `daily_budget_exceeded` / `http_error`）

- `status`：HTTP 状态码
- `message`：给人看的文案

第四步：分层定位

现象	大概率原因	怎么确认
「后端未连接」	服务没起 / 域名错了	curl `/api/health`
「收到的是 HTML 页面」	**网关错误页**，不是接口问题	看响应 content-type
「请求超时或已被取消」	跑太久或网络断	看是否 300s 超时
「额度用尽」	每日预算	看 `reset_at_ms`
页面白屏	组件崩溃	看 `ErrorBoundary` 有没有兜住

深挖追问：「HTML 错误页为什么要单独识别？」

> 因为它是最容易误判的一类。用户看到「后端连不上」会去重启后端， 但实际是**网关超时返回的 502 错误页**，后端好好地在那儿。 不区分这两者，排查方向完全错。我见过把「接口地址写错」和 「网关错误页」都报成同一句「请确认 API_BASE_URL」的情况—— 结果排查方向从一开始就跑偏了。

Q9.2 这个项目部署上有什么坑?

标准答法:

坑 1：配置文件静默失效

线上环境一直跑在开发配置上。两个独立缺陷叠加:

- `env_file` 用了**相对路径**，而部署环境的工作目录不是 `backend/`， 所有 `.env` 配置项**静默失效**（不报错，只是没生效）
- 多配置文件优先级互相污染（上传不按 `.gitignore` 过滤， 开发配置也上了服务器，且优先级更高）

→ 改成按运行环境**只读一个**配置文件。

> **测试视角的启示**：**「静默失效」比「直接报错」危险得多。** 报错至少会暴露问题；静默失效会让功能用错误的配置跑， 你还以为它在正常工作。所以配置项要有「生效值」的可见性—— 这个项目有 `GET /api/settings` 专门把当前生效的配置回显出来。

坑 2：单端口约束 托管环境只暴露一个 HTTP 端口，没有第二个给前端 dev server。 所以后端必须同时提供前端页面和 `/api`，SPA 深链回退到 `index.html`（含目录穿越防护）。

坑 3：数据被打包带走 运行时数据原本在 `backend/` 里，部署会把整个目录打包上传， 导致**本地数据覆盖线上数据**。→ 把运行时数据搬到项目根的 `data/`， 部署只上传后端代码。

坑 4：反向代理缓冲 SSE（前面 Q6.1 详述）

坑 5：多进程会破坏限流（前面 Q5.2 详述）

深挖追问：「怎么确保部署后配置真的生效了？」

> 这个项目有 `GET /api/llm/provider` 接口返回**当前生效的模型配置**， **但不含密钥**（只返回 provider 名、模型名、base_url）。 部署后第一件事就是调它，确认返回的是生产配置而不是开发配置。 这是个很实用的「部署后自检」思路。

备注：这题面试官很可能问「线上出过什么问题」。 讲这五个坑，每个都有具体的根因和处理，比泛泛说「部署过几次」强太多。

第 10 部分：反问面试官

准备 2–3 个，别问薪资和加班。

> **Q1**：这个项目现在的测试是怎么做的？有自动化测试的基建吗？ 目的是找到自己**能补位**的地方（如果他们缺自动化，我这段经验直接对口）。
>
> **Q2**：游戏测试的日常流程是怎样的？从需求到上线，一个版本大概要走几个环节？ 目的是了解**工作流和协作方式**（策划/开发/测试怎么分工，用什么工具， 提 bug 走什么流程）。
>
> **Q3**：线上出问题的时候，测试和开发是怎么分工的？ 目的是了解**线上问题的处理机制**（值守、报警、复盘）。
>
> **Q4（如果前面聊得深）**：你们的自动化测试是自己写框架还是用现成的 （比如 Airtest / Unity Test Framework / 自研 CI）？

收尾话术：

> 我在这个项目里做的最多的事情，其实就是**把「感觉没问题」变成「有断言能证明没问题」**—— 不管是接口的错误分支、状态机的非法转移，还是对比度和像素。 我理解游戏测试也是一样的：玩法逻辑对不对、手感对不对、 边界会不会崩，最终还是要有一堆能重复执行的验证手段。 我挺想看看你们那边是怎么做的。

附：面试前速记卡

必须记住的数字

项	数字
后端测试	225 个 test 函数 / 237 个用例通过
前端依赖	5 个（react / react-dom / motion / react-markdown / remark-gfm）
前端组件	22 个通用组件
页面	9 个
工作流节点	7 个业务阶段 + 1 个 `fail` 失败终态 = 8 个 `add_node`
颜色令牌	主题色全部集中在 `tokens.css`（158 处颜色值）；业务 CSS 仅 11 处硬编码，且都是渐变遮罩的技术性值（`#000` 遮罩、渐变上的 `#ffffff`）或 `var(--x, #fallback)` 兜底值
响应式断点	4 个（1200 / 1024 / 768 / 375）
对比度基线	浅底正文 ≥5.0:1，深底 ≥6.5:1，hover ≥4.5:1
SSE 看门狗	6000ms 不 open 转轮询；轮询 2500ms / 快照 3000ms
请求超时	300 秒
防失控四闸	max_iterations=5、max_verify_attempts=2、MAX_FAILURE_STREAK=2、recursion_limit=50
HTTP/1.1 并发连接上限	每域名 6 个（降级前必须关掉卡死的 SSE）

必须能白板画出来的流程

```text
START → understand_task → plan → research ↺ → retrieve → analyze → verify ↺
      → [中断] 等待人工批准 → write → END
```

三个「故事」准备好（每个 90 秒）

1. **断线重连与增量补发**（SSE + 轮询降级 + 事件落库）
2. **状态一致性**（重复批准 409、刷新不重弹确认框、终态不可复活）
3. **把视觉变成可断言**（WCAG 对比度实测 + 像素级采样 + 字体真加载检查）

三条测试原则（被问「你的测试理念」时用）

1. **测非法路径比测合法路径有价值** —— 状态机的 bug 全在非法转移里

2. **等「内容就绪」而不是「DOM 就绪」** —— 骨架屏会让就绪判断提前通过
3. **静默失效比直接报错更危险** —— 配置错了要能看见生效值

绝对不要说的话

- ❌「这个 bug 是我上一次没改好」—— 说「根因是 X，处理是 Y」
- ❌「我用了 AI 帮我写」—— 说「我用探针脚本自动化了验证」
- ❌「免费额度不够用」—— 说「有配额记账和降级机制」
- ❌「我在这台机器上跑的是 XX 模型」—— 说「支持多后端可切换」

本手册基于 commit `f5ea56d` 的实现整理。答案里的每个数字和文件名都可在仓库中核对。